SQL

Performing (Salary Analytics, Performance Analytics ,Strategic insights) Of Employees using Master Table, Employee Table , and performance table

(Master table)

```
create database db1

create table Departments(
dept_id int primary key,
dept_name varchar(50) not null unique
);
```

Description: Created Department Table with dept_id, dept_name

```
create table Locations(
location_id int primary key,
location_name varchar(50) not null
);
```

Description: Created Location Table with Location_id, Location Name

```
create table Job_Roles(
role_id int primary key,
role_name varchar(50) not null,
min_salary DECIMAL(10,2),
max_salary Decimal(10,2)
);
```

Description:CreatedJob_Roles Table with role_id,role_name,min_salary,max_salary

################ Insert data ####################################

```
Insert into Departments values
(1,'it'),(2,'hr'),(3,'Finance'),(4,'Marketing'),(5,'Operations');

Insert into Locations VALUES (1,'Bangalore')(2,'Pune'),(3,'Hyderabad');

Insert into Job_Roles VALUES
(1,'Analyst',300000,600000),
(2,'Senior Analyst',60000,30000),
(3,'Manager',60000,9000000)
(4,'Lead'',60000,120000000)
```

######################## Employees Table ##########################

```
create table Employees(
emp_id int primary key,
emp_name varchar(100) not null,
gender varchar(10),
dept_id int,
role_id int,
location_id  int,
join_date DATE,
status varchar(20) default 'Active',
FOREIGN KEY (dept_id) REFERENCES Departments(dept_id),
FOREIGN KEY (role_id) REFERENCES Job_Roles(role_id),
FOREIGN KEY (location_id) REFERENCES Locations(location_id))
INSERT INTO Employees VALUES
(101,'Amit Sharma','Male',1,1,1,'2021-06-15','Active'),
(102,'Priya Nair','Female',1,2,2,'2021-06-15','Active'),
```

Description: Created Employee Table with fields emp_id, emp_name, dept_id, role_id, location,join_date

```
create table Salaries(
salary_id int primary key,
emp_id int,
salary decimal(10,2),
effective_date DATE,
FOREIGN KEY (emp_id) REFERENCES Employees(emp_id));
```

Description:Created Salaries Table with salary_id,emp_id,salary,effective_date,

```
Insert into Salaries Values
(1,101,50000000,'2023-04-01'),
(2,102,750000,'2023-04-02')
```

########################## Performance Table #######################

```
create table Performance_Reviews(
review_id int primary key,
emp_id int,
review_year int,
rating decimal(2,1) check (rating between 1 and 5),
foreign key (emp_id) references Employees(emp_id)
);
```

Description: Created Performance Table with review_id, emp_id, review_year,

rating

Insert into Performance_Reviews values
(1,101,2023,4.2),
(2,102,2023,4.8)

-- Performance Analytics

-- 1. Top 10 high performing employees

select top 10 * from Performance_Reviews
order by rating desc

	review_id	emp_id	review_year	rating
1	8	108	2026	5.0
2	53	153	2018	5.0
3	15	115	2015	4.9
4	71	171	2019	4.9
5	96	196	2020	4.9
6	34	134	2015	4.8

-- 2 Average rating per department

```
SELECT
   d.dept_name,
   AVG(pr.rating) AS avg_rating
FROM
   Departments d
JOIN
   Employees e ON d.dept_id = e.dept_id
JOIN
   Performance_Reviews pr ON e.emp_id = pr.emp_id
GROUP BY
   d.dept_name
```

	dept_name	avg_rating
1	Data	2.915789
2	DevOps	2.878260
3	DotNET	2.935000
4	HR	3.094444
5	Java	2.730000

-- 3 Employees eligible for promotion (rating>4 & 2+ years)

```
SELECT
   e.emp_name,
   d.dept_name,
   pr.rating,
   e.join_date,
   DATEDIFF(YEAR, e.join_date, GETDATE()) AS years_with_company
FROM Employees e
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
JOIN Departments d ON e.dept_id = d.dept_id
WHERE pr.rating > 4.0
  AND DATEDIFF(YEAR, e.join_date, GETDATE()) > 2
ORDER BY pr.rating DESC;
```

	emp_name	dept_name	rating	join_date	years_with_company
1	Rohan Gupta	DevOps	5.0	2022-11-28	4
2	Amit Kumar	DevOps	5.0	2019-01-02	7
3	Arjun Iyer	Data	4.9	2023-06-24	3
4	Sneha Kumar	HR	4.9	2017-10-06	9
5	Rohan Patel	DotNET	4.9	2017-01-27	9
6	Rahul Gupta	DevOps	4.8	2017-12-21	9

— 4 performance trend over years

```
SELECT
    pr.review_year,
    AVG(pr.rating) AS avg_rating
FROM
    Performance_Reviews pr
GROUP BY
    pr.review_year
ORDER BY
    pr.review_year;
```

	review_year	avg_rating
1	2015	2.844444
2	2016	3.116666
3	2017	3.566666
4	2018	3.325000
5	2019	3.388888
6	2020	3.100000

-- Salary Analytics
-- 5. Salary distribution per department

```
SELECT
   d.dept_name,
   COUNT(s.salary_id) AS num_employees,
   AVG(s.salary) AS avg_salary,
   MIN(s.salary) AS min_salary,
   MAX(s.salary) AS max_salary
FROM
   Salaries s
JOIN
   Employees e ON s.emp_id = e.emp_id
JOIN
   Departments d ON e.dept_id = d.dept_id
GROUP BY
   d.dept_name
ORDER BY
   D.dept_name;
```

	dept_name	num_employees	avg_salary	min_salary	max_salary
1	Data	19	728421.052631	515000.00	995000.00
2	DevOps	23	752173.913043	500000.00	985000.00
3	DotNET	20	808000.000000	560000.00	980000.00
4	HR	18	691111.111111	510000.00	920000.00
5	Java	20	750500.000000	505000.00	990000.00

—6 -- Highest & lowest salary per role.

```
SELECT
   d.dept_name,
   COUNT(s.salary_id) AS num_employees,
   MIN(s.salary) AS min_salary,
   MAX(s.salary) AS max_salary
FROM
   Salaries s
JOIN
   Employees e ON s.emp_id = e.emp_id
JOIN
   Departments d ON e.dept_id = d.dept_id
GROUP BY
   d.dept_name
ORDER BY
```

```
    d.dept_name;
```

	dept_name	num_employees	min_salary	max_salary
1	Data	19	515000.00	995000.00
2	DevOps	23	500000.00	985000.00
3	DotNET	20	560000.00	980000.00
4	HR	18	510000.00	920000.00
5	Java	20	505000.00	990000.00

– 7 -- Salary vs performance correlation.

```
SELECT
    e.emp_name,
    d.dept_name,
    s.salary,
    pr.rating,
    CASE
        WHEN s.salary > (SELECT AVG(salary) FROM Salaries) AND pr.rating < 3.0
THEN 'Overpaid / Underperforming'
        WHEN s.salary < (SELECT AVG(salary) FROM Salaries) AND pr.rating > 4.0
THEN 'Underpaid / High Performer'
        ELSE 'Balanced'
    END AS pay_performance_status
FROM Employees e
JOIN Salaries s ON e.emp_id = s.emp_id
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
JOIN Departments d ON e.dept_id = d.dept_id
ORDER BY pr.rating DESC, s.salary DESC;
```

	emp_name	dept_name	salary	rating	pay_performance_status
1	Amit Kumar	DevOps	760000.00	5.0	Balanced
2	Rohan Gupta	DevOps	535000.00	5.0	Underpaid / High Performer
3	Rohan Patel	DotNET	975000.00	4.9	Balanced
4	Sneha Kumar	HR	850000.00	4.9	Balanced
5	Arjun Iyer	Data	570000.00	4.9	Underpaid / High Performer
6	Divya Rao	Data	955000.00	4.8	Balanced

8 –-- Employees below department average salary

```
WITH DeptAvg AS (
    SELECT
        e.emp_name,
```

```
        d.dept_name,
        s.salary,
        AVG(s.salary) OVER(PARTITION BY d.dept_id) AS dept_avg_salary
    FROM Employees e
    JOIN Salaries s ON e.emp_id = s.emp_id
    JOIN Departments d ON e.dept_id = d.dept_id
)
SELECT
    emp_name,
    dept_name,
    salary,
    dept_avg_salary,
    (dept_avg_salary - salary) AS salary_gap
FROM DeptAvg
WHERE salary < dept_avg_salary
ORDER BY dept_name, salary_gap DESC;
```

	emp_name	dept_name	salary	dept_avg_salary	salary_gap
1	Rohan Kumar	Data	515000.00	728421.052631	213421.052631
2	Manoj Rao	Data	520000.00	728421.052631	208421.052631
3	Divya Reddy	Data	545000.00	728421.052631	183421.052631
4	Arjun Iyer	Data	570000.00	728421.052631	158421.052631
5	Kavya Sharma	Data	590000.00	728421.052631	138421.052631
6	Neha Singh	Data	605000.00	728421.052631	123421.052631

-- Department Growth

-- 9 Hiring trend per year

```
SELECT
    YEAR(join_date) AS hire_year,
    COUNT(emp_id) AS total_new_hires
FROM Employees
GROUP BY YEAR(join_date)
ORDER BY hire_year ASC;
```

	hire_year	total_new_hires
1	2015	9
2	2016	9
3	2017	15
4	2018	3
5	2019	9
6	2020	13

--10 Headcount growth by department

```
SELECT
   d.dept_name,
   COUNT(e.emp_id) AS current_headcount
FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
ORDER BY current_headcount DESC;
```

	dept_name	current_headcount
1	DevOps	23
2	Java	20
3	DotNET	20
4	Data	19
5	HR	18

-- 11 Attrition rate by department

```
SELECT
   d.dept_name,
   COUNT(e.emp_id) AS total_hired,
   SUM(CASE WHEN e.status != 'Active' THEN 1 ELSE 0 END) AS employees_left,
   CAST(
      (SUM(CASE WHEN e.status != 'Active' THEN 1 ELSE 0 END) * 100.0) /
COUNT(e.emp_id)

      AS DECIMAL(5,2)
   ) AS attrition_rate_percentage
FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
ORDER BY attrition_rate_percentage DESC;
```

	dept_name	total_hired	employees_left	attrition_rate_percentage
1	HR	18	11	61.11
2	DotNET	20	12	60.00
3	Java	20	11	55.00
4	Data	19	10	52.63
5	DevOps	23	10	43.48

Strategic Insights

–.12 Employees overdue for promotion

```
---employees overdue for promotions
SELECT
   e.emp_name,
   d.dept_name,
   pr.rating,
   e.join_date,
   DATEDIFF(YEAR, e.join_date, GETDATE()) AS years_with_company,
   CASE
      WHEN pr.rating < 4.0 AND DATEDIFF(YEAR, e.join_date, GETDATE()) > 2
THEN 'Overdue for Promotion'
      ELSE 'Eligible for Promotion'
   END AS promotion_status
FROM Employees e
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
JOIN Departments d ON e.dept_id = d.dept_id
WHERE DATEDIFF(YEAR, e.join_date, GETDATE()) > 2
ORDER BY promotion_status DESC, pr.rating DESC;
```

	emp_name	dept_name	rating	join_date	years_with_company	promotion_status
1	Divya Reddy	Data	3.8	2019-06-24	7	Overdue for Promotion
2	Amit Singh	HR	3.8	2020-06-07	6	Overdue for Promotion
3	Rahul Singh	DotNET	3.7	2021-04-23	5	Overdue for Promotion
4	Divya Iyer	Data	3.7	2022-09-22	4	Overdue for Promotion
5	Kiran Reddy	DotNET	3.6	2016-07-30	10	Overdue for Promotion
6	Divya Sharma	DotNET	3.6	2017-10-19	9	Overdue for Promotion

--13 High salary but low performance cases

```
SELECT
   e.emp_name,
   d.dept_name,
   jr.role_name,
   s.salary,
   pr.rating,
   (SELECT AVG(salary) FROM Salaries) AS company_avg_salary
FROM Employees e
JOIN Salaries s ON e.emp_id = s.emp_id
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
JOIN Departments d ON e.dept_id = d.dept_id
JOIN Job_Roles jr ON e.role_id = jr.role_id
WHERE s.salary > (SELECT AVG(salary) FROM Salaries)
  AND pr.rating < 3.0
ORDER BY s.salary DESC;
```

	emp_name	dept_name	role_name	salary	rating	company_avg_salary
1	Rohan Rao	Java	Manager	990000.00	2.8	747500.000000
2	Meera Rao	DevOps	Associate	985000.00	2.6	747500.000000
3	Naveen Gupta	DotNET	HR	980000.00	1.2	747500.000000
4	Swathi Rao	DevOps	Data Scientist	970000.00	1.6	747500.000000
5	Amit Kumar	DevOps	Data Scientist	965000.00	1.5	747500.000000
6	Priya Patel	Java	HR	960000.00	2.4	747500.000000

---14 most stable department(lowest attrition)

```
SELECT TOP 1
   d.dept_name,
   COUNT(e.emp_id) AS total_ever_hired,
   SUM(CASE WHEN e.status = 'Active' THEN 1 ELSE 0 END) AS current_active_employees,
   SUM(CASE WHEN e.status != 'Active' THEN 1 ELSE 0 END) AS departures,
   CAST(
      (SUM(CASE WHEN e.status != 'Active' THEN 1 ELSE 0 END) * 100.0) / NULLIF(COUNT(e.emp_id), 0)
      AS DECIMAL(5,2)
   ) AS attrition_rate
FROM Departments d
LEFT JOIN Employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name
ORDER BY attrition_rate ASC;
```

	dept_name	total_ever_hired	current_active_employees	departures	attrition_rate
1	DevOps	23	13	10	43.48

---16 location-wise salary comparision

```
SELECT
   l.location_name,
   COUNT(e.emp_id) AS employee_count,
   MIN(s.salary) AS min_salary,
   MAX(s.salary) AS max_salary,
   AVG(s.salary) AS avg_location_salary
FROM Locations l
JOIN Employees e ON l.location_id = e.location_id
JOIN Salaries s ON e.emp_id = s.emp_id
GROUP BY l.location_name
```

ORDER BY avg_location_salary DESC;

select * from Employees

	location_name	employee_count	min_salary	max_salary	avg_location_salary
1	HR	30	500000.00	995000.00	772000.000000
2	Data	36	505000.00	970000.00	743194.444444
3	DevOps	34	515000.00	985000.00	730441.176470

--- 17 gender diversity ratio (if added)

select gender,count(gender) as total
from Employees
group by gender

SELECT
gender,
COUNT(*) AS headcount,
CAST(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER() AS DECIMAL(5,2)) AS diversity_percentage
FROM Employees
GROUP BY gender;

	gender	total
1	Female	52
2	Male	48

	gender	headcount	diversity_percentage
1	Female	52	52.00
2	Male	48	48.00

---18 Median salary calculation

SELECT DISTINCT
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) OVER() AS median_salary
FROM Salaries;

SELECT DISTINCT
d.dept_name,
PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY s.salary) OVER(PARTITION BY d.dept_name) AS median_dept_salary
FROM Departments d

```
JOIN Employees e ON d.dept_id = e.dept_id
JOIN Salaries s ON e.emp_id = s.emp_id
ORDER BY median_dept_salary DESC;
```

	median_salary
1	747500

	dept_name	median_dept_salary
1	DotNET	830000
2	DevOps	760000
3	Java	760000
4	Data	720000

--- 20 Identify leadership pipeline candidates

```
SELECT
   e.emp_name,
   d.dept_name,
   jr.role_name AS current_role,
   pr.rating AS last_performance_score,
   DATEDIFF(YEAR, e.join_date, '2026-03-03') AS tenure_years,
   s.salary,
   CASE
      WHEN s.salary >= (jr.max_salary * 0.90) THEN 'Ready for Grade Jump'
      ELSE 'Developmental Lead'
   END AS pipeline_status
FROM Employees e
JOIN Departments d ON e.dept_id = d.dept_id
JOIN Job_Roles jr ON e.role_id = jr.role_id
JOIN Performance_Reviews pr ON e.emp_id = pr.emp_id
JOIN Salaries s ON e.emp_id = s.emp_id
WHERE pr.rating >= 4.5
  AND e.status = 'Active'
  AND DATEDIFF(YEAR, e.join_date, '2026-03-03') >= 3
ORDER BY pr.rating DESC, tenure_years DESC;
```

	emp_name	dept_name	current_role	last_performance_score	tenure_years	salary	pipeline_status
1	Rohan Gupta	DevOps	SDE 1	5.0	4	535000.00	Developmental Lead
2	Arjun Iyer	Data	Data Scientist	4.9	3	570000.00	Ready for Grade Jump
3	Swathi Rao	DotNET	Data Scientist	4.5	9	575000.00	Ready for Grade Jump
4	Kavya Sharma	DevOps	Associate	4.6	10	580000.00	Ready for Grade Jump
5	Naveen Rao	DotNET	Data Scientist	4.8	3	665000.00	Ready for Grade Jump
6	Pooja Rao	HR	HR	4.7	3	705000.00	Ready for Grade Jump